QP Code - 25901229

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

MGU-UGP (HONOURS) REGULAR EXAMINATION March 2026

Fourth SEMESTER

Discipline Specific Elective (DSE) - MG4DSEECT200 - ARM BASED EMBEDDED SYSTEMS

(2024 ADMISSION ONWARDS)

Duration: 2 hours Maximum Marks: 70

Part A MCQ Answer all questions

Each question carries 1 marks

1. d). Mapping variables to CPU registers
2. c) Out-of-order completion
3. a) Hardware-dependent
4. b) pinMode()
5. a) BNE
6. b) Avoid floating input
7. **b) R15**
8. b) Single-master multi-slave
9. c) ARM Cortex-M
10. c) Microcontroller applications
11. **b) Special Function Registers (SFRs)**
12. a) System
13. d) All of the above
14. c) Reset
15. b) More hardware and control logic
16. c) Pipelining

Part B Short Answer Questions

Answer any 6 questions Each question carries 5 marks

17. **Latency:**

- **FIQ (Fast Interrupt Request):** Lower latency → faster response (handled with higher priority and minimal overhead)
- **IRQ (Interrupt Request):** Higher latency compared to FIQ

Register Usage:

- **FIQ:** Has more **banked registers** (R8–R14), so it can operate without saving/restoring many registers → faster execution
- **IRQ:** Fewer banked registers (only R13–R14), so it often needs to save/restore registers → slower than FIQ

18. Any five give 5 mark

Real-Time Operation

Embedded systems must respond within strict time constraints.

- **Hard real-time:** Missing a deadline = system failure
- **Soft real-time:** Occasional delays are acceptable

Reliability

Systems should operate continuously without failure.

- Important in critical applications like automotive, medical devices
- Requires fault tolerance and robust design

Efficiency (Performance & Power)

- Must use minimal resources (CPU, memory)
- Low power consumption is crucial, especially for battery-powered devices

Cost Effectiveness

- Hardware and software should be optimized to reduce overall cost
- Selection of components is based on budget and requirements

Compact Size

- Embedded systems are often part of small devices
- Requires minimal hardware footprint and optimized PCB design

Application-Specific Design

- Designed for a specific task (not general-purpose)
- Improves performance and efficiency

Scalability and Flexibility

- Design should allow future upgrades or modifications
- Modular approach helps reuse components

Hardware-Software Co-Design

- Hardware and software are developed together for optimal performance
- Trade-offs are made between hardware complexity and software flexibility

Low Power Consumption

- Essential for portable and IoT devices
- Techniques include sleep modes, power gating

Robustness

- Must handle environmental conditions (temperature, noise, vibration)
- Includes error detection and recovery mechanisms

19.

1. Initialization

- Configure **GPIO pins**:
  - Sensor pin → INPUT
  - LED pin → OUTPUT
  - Buzzer pin → OUTPUT
- Set initial states:
  - LED OFF
  - Buzzer OFF

2. Continuous Monitoring (Main Loop)

- Continuously read the sensor input (e.g., motion, temperature, gas, etc.)

3. Condition Check

- Compare the sensor value with a predefined threshold or condition:
  - Example:
    - Motion detected
    - Temperature exceeds limit
    - Gas leakage detected

4. Trigger Alert

- If condition is **TRUE**:
  - Turn ON LED (visual indication)
  - Turn ON buzzer (audio alert)

5. Reset / Normal State

- If condition is **FALSE**:
  - Turn OFF LED
  - Turn OFF buzzer

20.

Feature	CPSR	SPSR
Full form	Current Program Status Register	Saved Program Status Register
Purpose	Holds current CPU status	Saves previous CPU status during exception
Availability	Always present	Only in exception modes
Usage	Active execution	Used for restoring state after interrupt
Copies	Single	One per exception mode

21. **Register allocation in Embedded C for ARM** refers to the process of **assigning program variables to CPU registers instead of memory** to improve execution speed.

Explanation

- ARM processors have a limited set of **general-purpose registers** (e.g., R0–R12).
- During compilation, the compiler decides which variables should be stored in these registers.
- Accessing registers is **much faster** than accessing RAM, so this improves

22. The **ARM Cortex-M memory map** defines how the **entire address space (4 GB)** is organized into different regions for code, data, and peripherals.

Total Address Space

- Cortex-M uses a **32-bit address space → 4 GB (0x00000000 to 0xFFFFFFFF)**

Address Range	Region	Description
0x00000000 – 0x1FFFFFFF	**Code Region**	Program memory (Flash), interrupt vectors
0x20000000 – 0x3FFFFFFF	**SRAM Region**	Data memory (RAM)
0x40000000 – 0x5FFFFFFF	**Peripheral Region**	Memory-mapped I/O registers
0x60000000 – 0x9FFFFFFF	**External RAM**	External memory (if available)
0xA0000000 – 0xDFFFFFFF	**External Devices**	External peripherals
0xE0000000 – 0xFFFFFFFF	**System Region**	NVIC, system control, debug components

1. Code Region

- Stores program instructions (Flash memory)
- Contains **interrupt vector table** at the start

2. SRAM Region

- Used for variables, stack, and heap
- Fast read/write memory

3. Peripheral Region

- Contains registers for GPIO, UART, timers, etc.
- Accessed using **memory-mapped I/O**

4. External RAM

- Optional external memory expansion

5. External Device Region

- Used for interfacing external hardware devices

6. System Region

- Contains:
  - NVIC (interrupt controller)
  - SysTick timer
  - Debug and control registers

23. ODR (Output Data Register)

- Used to **write data to GPIO pins**
- Controls the **output state** (HIGH or LOW)
- Applicable when pin is configured as **OUTPUT**
- Writing:
  - `1` → sets pin HIGH
  - `0` → sets pin LOW

IDR (Input Data Register)

- Used to **read data from GPIO pins**
- Reflects the **current logic level** at the pin
- Applicable when pin is configured as **INPUT**
- Reading:
  - `1` → pin is HIGH
  - `0` → pin is LOW

Feature	ODR	IDR
Full form	Output Data Register	Input Data Register
Operation	Write	Read
Purpose	Control output pins	Read input pins
Direction	CPU → Pin	Pin → CPU
Usage mode	Output mode	Input mode

24. The `volatile` **keyword** in Embedded C is used to **tell the compiler that a variable's value can change unexpectedly**, outside the normal program flow. This prevents the compiler from applying certain optimizations that assume the value doesn't change on its own.

Purpose

- Ensures the **compiler always reads the variable from memory**, rather than using a cached value in a register.
- Commonly used for:
  - **Hardware registers** (GPIO, timers, ADCs)
  - **Global variables modified in interrupts**
  - **Shared memory in multi-threaded or RTOS applications**

Part C Essay Questions

Answer any 2 questions Each question carries 12 marks

25. Describe any embedded system with diagram -5 mark
Classification – 5 mark

Classification of Embedded Systems

Embedded systems can be classified based on **functionality** and **performance**.

A. Based on Functionality

1. **Stand-alone Embedded Systems**
   - Operate independently without host computer
   - Example: Digital watches, MP3 players
2. **Real-Time Embedded Systems**
   - Provide outputs within **strict timing constraints**
   - Example: Airbag system in cars, pacemakers
3. **Networked Embedded Systems**
   - Communicate with other systems or networks
   - Example: Smart home devices, IoT sensors
4. **Mobile Embedded Systems**
   - Portable devices, usually battery-operated
   - Example: Smartphones, wearable fitness devices

B. Based on Performance

1. **High-Performance Embedded Systems**
   - Require **fast computation and high memory**
   - Example: Graphics processing in gaming consoles
2. **Medium-Performance Embedded Systems**

- Moderate processing for control tasks
- Example: Industrial automation controllers

3. **Low-Performance Embedded Systems**
   - Simple control, minimal resources
   - Example: Digital clocks, basic toys

26. **Flow chart /Steps ARM Exception – give 5 mark**
    Sequence of events during exception entry and exception return. – 5 Mark

ARM processors handle exceptions (interrupts, faults, errors) by switching execution to predefined **exception vectors** and saving the current state so execution can later resume.

? Common ARM Exceptions

- Reset
- Undefined Instruction
- Software Interrupt (SWI/SVC)
- Prefetch Abort
- Data Abort
- IRQ (Interrupt Request)
- FIQ (Fast Interrupt Request)

Flow Diagram/Steps of Exception Handling

- Exception Occur
- Processor Completes Current Instruction
- Save Return Address in LR_<mode>
- Save CPSR → SPSR_<mode>
- Change Processor Mode
- (IRQ, FIQ, Abort, Supervisor, etc.)
- Disable Interrupts (if required)
- Load PC with Exception Vector Address
- Execute Exception Handler
- Restore Context (Return)
- Resume Main Program

Exception Entry: Sequence of Events

When an exception occurs, ARM performs the following steps automatically:

1. Finish Current Instruction

- The processor completes execution of the current instruction (in most cases).

2. Save Return Address

- The **Program Counter (PC)** is adjusted and stored in the **Link Register (LR)** of the exception mode.
    - Example:
        - IRQ: `LR_irq = PC + 4`
        - Data Abort: `LR_abt = PC + 8`

3. Save Processor State

- The current **CPSR (Current Program Status Register)** is copied into:
    - **SPSR_<mode> (Saved Program Status Register)**

4. Change Processor Mode

- Processor switches to a privileged mode:
    - IRQ → IRQ mode
    - FIQ → FIQ mode
    - SWI → Supervisor mode

5. Disable Interrupts

- Depending on exception:
    - IRQ disabled for IRQ handler
    - Both IRQ & FIQ disabled for FIQ

6. Set PC to Exception Vector

- PC is loaded with a fixed address from the vector table:

Exception Return: Sequence of Events

After servicing the exception, control returns to the interrupted program.

1. Restore CPSR

- The saved SPSR is restored to CPSR:

```
CPSR ← SPSR_<mode>
```

2. Restore PC

- Return address is restored from LR:

```
PC ← LR_<mode> (with adjustment)
```

3. Use Special Return Instruction

- Common instructions:
  - `SUBS PC, LR, #4`
  - `MOVS PC, LR`
- The `S` suffix ensures:
  - CPSR is restored from SPSR simultaneously.

4. Resume Execution

- Processor resumes execution of the interrupted program.

27. working principle of an IR sensor – 4 mark
Interfacing of an IR sensor with ARM Cortex M – 6 mark

Working Principle of an IR Sensor

An **Infrared (IR) sensor** is commonly used for **object detection** and **proximity sensing**.

Basic Components

- **IR LED (Transmitter)** → emits infrared light
- **Photodiode / Phototransistor (Receiver)** → detects reflected IR light
- Signal conditioning circuit (Comparator/Amplifier)

Principle of Operation

✔ Step-by-step working:

1. **IR Emission**
   - IR LED emits infrared radiation continuously.
2. **Reflection from Object**
   - When an object comes near, IR rays reflect back.
3. **Detection**
   - The photodiode/phototransistor detects reflected IR light.
4. **Signal Conversion**
   - Light intensity → electrical signal (current/voltage).
5. **Output Generation**
   - Comparator converts analog signal into:
     - **Digital output (HIGH/LOW)** or
     - **Analog voltage (proportional to distance)**

Types of Output

- **Digital IR Sensor**
  - Output: HIGH/LOW
  - Used for obstacle detection
- **Analog IR Sensor**
  - Output: variable voltage
  - Used for distance measurement

Interfacing IR Sensor with ARM Cortex-M

Typical ARM Cortex-M (e.g., STM32) Pins Used

- GPIO (for digital input)
- ADC (for analog input)
- Power pins (VCC, GND)

Hardware Connections

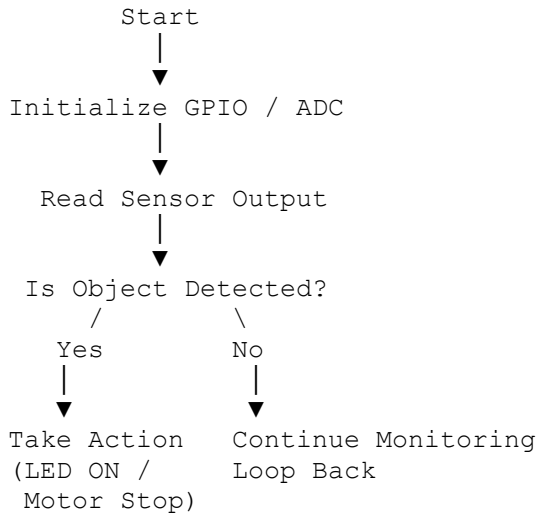
Case 1: Digital IR Sensor Interface

```
IR Sensor         ARM Cortex-M
---------         -------------
VCC      ──────►  3.3V
GND      ──────►  GND
OUT      ──────►  GPIO Input Pin (e.g., PA0)
```

Case 2: Analog IR Sensor Interface

```
IR Sensor         ARM Cortex-M
---------         -------------
VCC      ──────►  3.3V
GND      ──────►  GND
OUT      ──────►  ADC Pin (e.g., PA1)
```

Flow Diagram of Operation



Software Steps (Cortex-M)

For Digital Sensor

1. Configure GPIO pin as **input**

2. Continuously read pin status
3. Check:
    - LOW → Object detected
    - HIGH → No object

For Analog Sensor

1. Configure ADC channel
2. Read analog value
3. Compare with threshold

28. Interfacing Seven-segment Display – 6 mark
Program -4 mark